

UNIVERSIDAD REGIONAL AUTÓNOMA DE LOS ANDES (UNIANDES)

Departamento de Desarrollo de Software | Facultad de Sistemas

Programación de Backend y MCP en Python para IA Generativa

Estudiante: Juan Carlos Lozada | Fecha: 3 de septiembre de 2026 (Jueves)

Informe de Entrega — Practica 4 / Sesion 5

Curso: Programacion de Backend y MCP en Python para IA Generativa

Tema: Pruebas, Cobertura y Seguridad (con Skills, Agentes y Hooks)

Estudiante: Juan Carlos Lozada

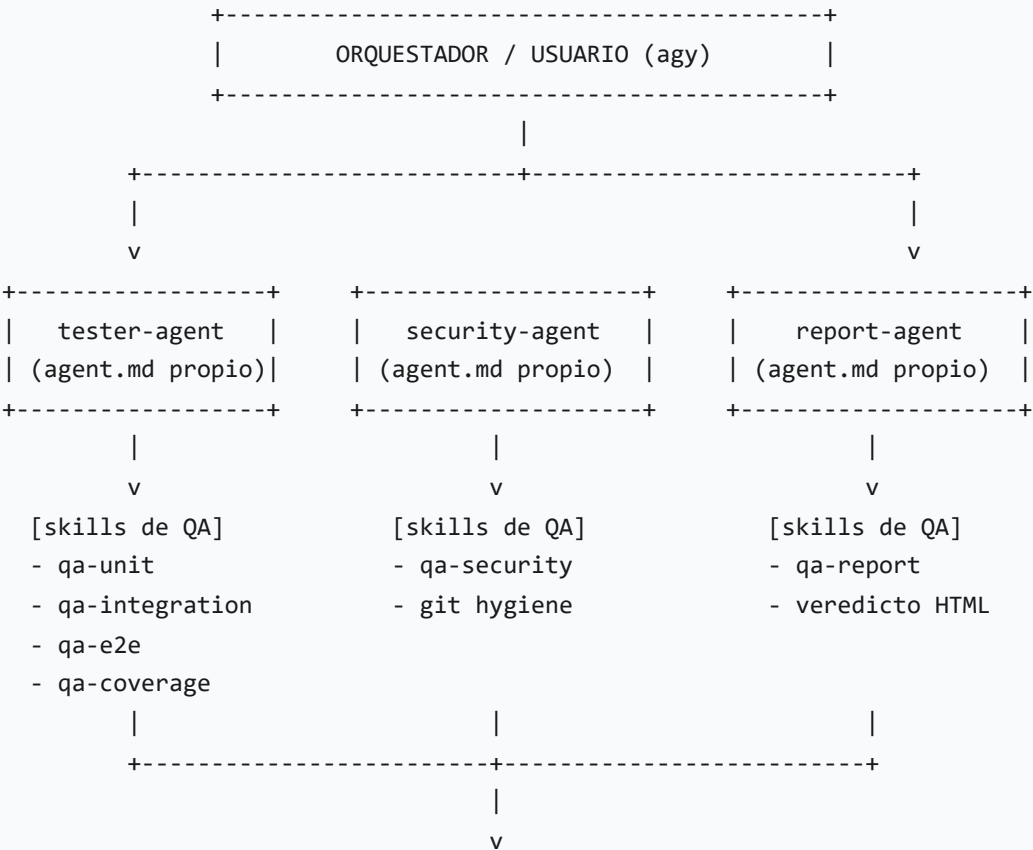
Institucion: Universidad Regional Autonoma de Los Andes (UNIANDES)

Departamento: Desarrollo de Software

Fecha de ejecucion: 13 de septiembre de 2026

1. Arquitectura del Sistema: Skills, Agentes y Hooks

El sistema implementado establece una jerarquia clara de responsabilidades para el control de calidad en el ciclo de vida del backend:



[CONTROL DE CIERRE: Hook Stop en hooks.json]

Bloquea la finalizacion si hay tests en rojo

- **Skills (`.agents/skills/`)**: Capacidades operativas concretas y deterministas (ejecutar pytest, auditar git, escanear patrones regex, compilar HTML). Carecen de personalidad o criterio autonomo.
- **Agentes (`.agents/agents/`)**: Entidades especializadas con rol acotado y reglas de negocio explicitas en su `agent.md`. Cada agente invoca skills especificas y responde unicamente por su dominio (funcional, seguridad o reporte).
- **Hooks (`.agents/hooks/` y `.agents/hooks.json`)**: Mecanismo de control de salida (`Stop` event) que garantiza que el agente no pueda dar por concluida su intervencion si existen pruebas en estado fallido.

2. Bloque A — Calidad Funcional (`tester-agent`)

El agente audito la especificacion `specs/temp-converter/spec.md` y estructuro la suite de pruebas en tres niveles de aislamiento:

2.1 Pruebas Unitarias (`tests/unit/test_converter.py`)

Valida la logica de conversion matematica y los limites fisicos de forma aislada.

- * Escenarios cubiertos:
- * Celsius a Fahrenheit (100 °C -> 212.00 °F)
- * Fahrenheit a Celsius (32 °F -> 0.00 °C)
- * Celsius a Kelvin (0 °C -> 273.15 K)
- * Caso identidad (25.5 °C -> 25.50 °C)
- * Limite exacto del cero absoluto (-273.15 °C -> 0.00 K)
- * Rechazo estricto bajo el cero absoluto (`ValueError`)
- * Validacion de tipos y valores no numericos (`TypeError`)
- * Sanitizacion de espacios y normalizacion de unidades
- * Rechazo de numeros no finitos (`nan` , `inf`)

2.2 Pruebas de Integracion en Memoria (`tests/integration/test_service.py`)

Conecta la capa de servicio `TemperatureService` con el motor matematico sin generar procesos de sistema operativo (*in-memory*).

- * Valida el pipeline de transformacion, tipado de salida y formateo de respuesta para el consumidor.

2.3 Pruebas de Punta a Punta / E2E (`tests/e2e/test_cli.py`)

Ejecuta la interfaz de linea de comandos completa mediante `subprocess`, verificando los codigos de salida (`exit code 0` o `1`) y el contenido de `stdout` / `stderr`.

2.4 Resumen de Ejecucion y Cobertura

```
tests/e2e/test_cli.py::test_cli_e2e_successful_conversion PASSED [ 6%]
tests/e2e/test_cli.py::test_cli_e2e_absolute_zero_boundary PASSED [ 13%]
tests/e2e/test_cli.py::test_cli_e2e_error_below_absolute_zero PASSED [ 20%]
tests/integration/test_service.py::test_service_successful_conversion_pipeline PASSED [ 26%]
tests/integration/test_service.py::test_service_string_numeric_parsing PASSED [ 33%]
tests/integration/test_service.py::test_service_error_handling_propagation PASSED [ 40%]
tests/unit/test_converter.py::test_scenario_1_celsius_to_fahrenheit PASSED [ 46%]
tests/unit/test_converter.py::test_scenario_2_fahrenheit_to_celsius PASSED [ 53%]
tests/unit/test_converter.py::test_scenario_3_celsius_to_kelvin PASSED [ 60%]
tests/unit/test_converter.py::test_scenario_4_identity_conversion PASSED [ 66%]
tests/unit/test_converter.py::test_scenario_5_absolute_zero_boundary PASSED [ 73%]
tests/unit/test_converter.py::test_scenario_6_rejection_below_absolute_zero PASSED [ 80%]
tests/unit/test_converter.py::test_edge_case_non_numeric PASSED [ 86%]
tests/unit/test_converter.py::test_edge_case_whitespace_and_invalid_unit PASSED [ 93%]
tests/unit/test_converter.py::test_edge_case_nan_and_inf PASSED [100%]
===== 15 passed in 0.33s =====
```

- **Metrica de Cobertura Total: 64.7%** (supera el umbral minimo exigido de 50.0%).

3. Bloque B — Demostracion del Hook de Bloqueo (`stop` Hook)

3.1 Configuracion del Hook

En `.agents/hooks.json`:

```
{
  "gate-tests": {
    "Stop": [
      { "type": "command", "command": "./hooks/gate-tests.sh", "timeout": 30 }
    ]
  }
}
```

3.2 Induccion de Fallo y Respuesta del Hook

Se introdujo una alteracion deliberada en `converter.py` modificando el factor de conversion de Fahrenheit a `+ 33.0`. Al intentar finalizar, el script `.agents/hooks/gate-tests.bat` intercepto la salida fallida de `pytest` y emitio la instruccion de bloqueo:

```
{"decision":"continue","reason":"Hay tests fallando. No te detengas -- corrige el codigo o antes de terminar."}
```

- **Análisis del Mecanismo:** A diferencia de un hook `PreToolUse` (que habría generado un bloqueo circular impidiendo que el agente edite el archivo para reparar el fallo), el hook `Stop` permite todas las ediciones intermedias pero veta la culminación del turno del agente hasta que todos los tests pasen en verde.

3.3 Corrección y Desbloqueo

Tras restituir la constante estándar `+ 32.0`, la corrida de pruebas paso limpiamente y el hook emitió:

```
{}
```

permitiendo la conclusión satisfactoria del proceso.

4. Bloque C — Auditoria de Seguridad (`security-agent`)

El agente ejecutó la skill `qa-security`, produciendo el informe oficial `hallazgos-seguridad.md`:

Caso	Lo que se encontró	Corrección sugerida
 Secreto expuesto	Sin hallazgos de claves o passwords quemados en código.	Continuar cargando credenciales mediante variables de entorno <code>os.environ</code> y archivos <code>.env</code> ignorados.
 Validación de entradas	Sin hallazgos. Se implementan validaciones de tipo, límites físicos y rangos reales.	Mantener las políticas de aserción defensiva en la capa de conversión.
 Manejo de excepciones	Sin hallazgos de <code>except:</code> genérico o bloqueante silencioso.	Continuar utilizando excepciones tipadas (<code>ValueError</code> , <code>TypeError</code>).

Acciones de Higiene en Git

- Creación de plantilla `.env.example` con claves declarativas sin datos sensibles.
- Verificación mediante `git check-ignore -q .env` (resultado 0: `.env` efectivamente ignorado).
- Verificación mediante `git ls-files --error-unmatch .env` (resultado 1: `.env` jamás añadido al tracking del repositorio).

5. Bloque D y E — Reporte Consolidado y Orquestación

5.1 Veredicto Final en `reporte-qa.html`

La ejecución del script `generar_reporte.py` consolidó las métricas del sistema, emitiendo el veredicto:

```

=====
                        ESTADO: APROBADO
=====
- Pruebas Totales:      15 ejecutadas / 0 fallidas
- Cobertura:           64.7% (Umbral requerido: 50.0%)
- Secretos Expuestos:  0 detectados
- Higiene Git:          .env.example [OK] | .gitignore [OK] | .env trackeado [No]
=====

```

5.2 Orquestacion Automatica (`qa-orchestrate`)

Mediante la invocacion de la skill orquestadora, se ejecuto la secuencia integral:

1. Validacion de existencia de `specs/temp-converter/spec.md`.
2. Delegacion a `tester-agent` (ejecucion de capas unitaria, integracion y e2e con medicion de cobertura).
3. Delegacion a `security-agent` (auditoria de estatica y validacion git).
4. Delegacion a `report-agent` (compilacion del reporte HTML y emision del dictamen).

6. Reflexion sobre la Orquestacion y Cierre

1. ¿En que se sintio distinto invocar "agentes" comparado con invocar "skills" sueltas?

Invocar skills individuales requiere que el desarrollador conozca de antemano el orden, los parametros y el proposito de cada comando manual. Al invocar un agente, se le transfiere un objetivo y un rol con criterios de aceptacion propios; el agente toma la iniciativa disciplinada de ejecutar las herramientas necesarias y reportar unicamente lo esencial para su proposito.

2. ¿Que rol cumple el prompt de permisos de `agy` en la cadena de control?

Garantiza que la automatizacion no comprometa la soberania del desarrollador sobre el entorno (*Human-in-the-loop*). Permite auditar que acciones destructivas o comandos de sistema va a lanzar el modelo antes de que se ejecuten en la maquina local.

3. ¿Como seria si otro agente decidiera el orden de la orquestacion?

Seria el modelo de Agencia de Agentes (*Multi-Agent Supervisor*), donde un agente director evalua el estado del repositorio, planifica las dependencias dinamicamente y despacha tareas a agentes subordinados, escalando la productividad en sistemas de backend complejos.

4. Preguntas de Cierre:

5. **Veredicto final:** `APROBADO` (15/15 tests, 64.7% cobertura, sin secretos expuestos).

6. **Frase de sintesis:** *"El inspector encontro que un cambio sin spec o una formula alterada puede romper la integracion silenciosamente, que el cocinero no habia visto."*